

Lab 4

Evaluation Question:

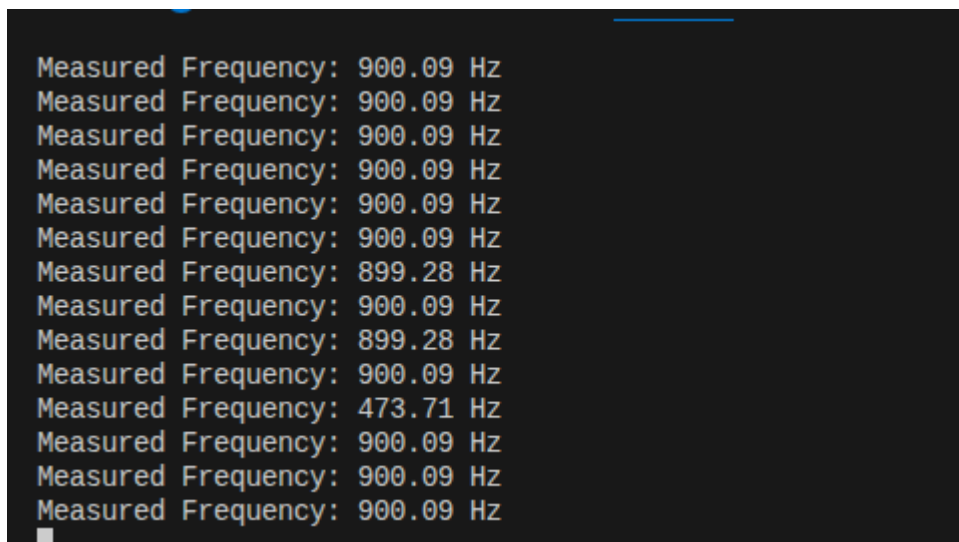
(a) What is the advantage of using a hardware timer for delays instead of a for or while loop?

The primary advantage of using a hardware timer over a software "busy-wait" loop (like `for(int i=0; i<1000; i++)`) is **deterministic timing** and **CPU efficiency**.

- **Software Loops:** The time a software loop takes depends on the CPU clock speed, compiler optimization levels, and whether the CPU is interrupted by other tasks. If you change your compiler settings from "Debug" to "Release," your software delay might suddenly become 10x faster because the compiler optimized the "useless" loop away.
- **Hardware Timers:** These are independent peripherals. Once configured, they count at a fixed rate dictated by the hardware clock, regardless of what the CPU is doing or how the code is compiled.
- **Software Loops:** The CPU is "trapped" inside the delay loop. It cannot perform other calculations or respond to other inputs until the loop finishes.
- **Hardware Timers:** By using interrupts (as we have done in Task 3), the CPU only spends a few microseconds inside the `Callback` function every 1ms. The rest of the time, the CPU is free to run the `main` loop, process sensor data, or even go into a low-power sleep mode to save battery.

Using a hardware timer allows the microcontroller to enter **Low Power Modes**. You can set a timer to wake the CPU up only when the delay is over. With a software loop, the CPU must stay fully powered and clocked at high speed just to count to a million.

Task 3 Extension Measured Freq SS:



```
Measured Frequency: 900.09 Hz
Measured Frequency: 900.09 Hz
Measured Frequency: 900.09 Hz
Measured Frequency: 900.09 Hz
Measured Frequency: 900.09 Hz
Measured Frequency: 900.09 Hz
Measured Frequency: 899.28 Hz
Measured Frequency: 900.09 Hz
Measured Frequency: 899.28 Hz
Measured Frequency: 900.09 Hz
Measured Frequency: 473.71 Hz
Measured Frequency: 900.09 Hz
Measured Frequency: 900.09 Hz
Measured Frequency: 900.09 Hz
```

Task 1 : As we want a output time of 1ms, and out System clock is 48Mhz.

If we can decrease the frequency to 1kHz, we will get an output time of 1ms

To set the output desired frequency we can use the formula : $f_{out} = \frac{sysClk}{(PSC+1)(Arr+1)}$

$$f_{out} = 1kHz$$

$$1kHz = \frac{48MHz}{(PSC+1)(Arr+1)}$$

$$(PSC + 1)(Arr + 1) = \frac{48M}{1k}$$

$$(PSC + 1)(Arr + 1) = 48000$$

We can make $(PSC + 1) = 48$ and $(Arr + 1) = 1000$

Hence $Arr = 999$ and $PSC = 47$

```
/* USER CODE END TIM2_Init 1 */
htim2.Instance = TIM2;
htim2.Init.Prescaler = 47;
htim2.Init.CounterMode = TIM_COUNTERMODE_UP;
htim2.Init.Period = 999;
htim2.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;
htim2.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_DISABLE;
if (HAL_TIM_Base_Init(&htim2) != HAL_OK)
{
    Error_Handler();
}
```

Code:

```
void delay_ms(uint32_t ms){
    __HAL_TIM_SET_COUNTER(&htim2, 0);

    HAL_TIM_Base_Start(&htim2);

    for (uint32_t i = 0; i < ms; i++) {
        while(__HAL_TIM_GET_COUNTER(&htim2) < 900) {
            // Waste time cuz Polling
        }

        __HAL_TIM_SET_COUNTER(&htim2, 0);
    }
    HAL_TIM_Base_Stop(&htim2);
}
```

```

while (1)
{
    /* USER CODE END WHILE */
    HAL_GPIO_TogglePin(GPIOE, GPIO_PIN_8);
    delay_ms(1000);
    /* USER CODE BEGIN 3 */
}

```

Task 2:

As we want 1 sec interrupts, we need a Frequency of 1 Hz

Basil's Approach:

$$1Hz = \frac{48Mhz}{(PSC+1)(Arr+1)}$$

$$(PSC + 1)(Arr + 1) = \frac{48Mhz}{1Hz}$$

$$(PSC + 1)(Arr + 1) = \frac{48 \cdot 10^6}{1}$$

$$(PSC + 1)(Arr + 1) = 48 \cdot 10^6$$

Let $(PSC + 1) = 48$ hence $PSC = 47$

Let $(Arr + 1) = 1000000$ hence $Arr = 999999$

Ruqaiya's Approach:

$$1Hz = \frac{48Mhz}{(PSC+1)(Arr+1)}$$

$$(PSC + 1)(Arr + 1) = \frac{48Mhz}{1Hz}$$

$$(PSC + 1)(Arr + 1) = \frac{48 \cdot 10^6}{1}$$

$$(PSC + 1)(Arr + 1) = 48 \cdot 10^6$$

Assume $Arr = 999$ Hence $(Arr + 1) = 1000$

$$(PSC + 1) 1000 = 48 \cdot 10^6$$

$$(PSC + 1) = 48 \cdot 10^3$$

$$(PSC) = 48000 - 1$$

$$(PSC) = 47999$$

```

/* USER CODE END TIM2_Init 1 */
htim2.Instance = TIM2;
htim2.Init.Prescaler = 47;
htim2.Init.CounterMode = TIM_COUNTERMODE_UP;
htim2.Init.Period = 999999;
htim2.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;
htim2.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_DISABLE;
if (HAL_TIM_Base_Init(&htim2) != HAL_OK)
{
    Error_Handler();
}

```

Task 3:

As we want 1ms it means we need 1 kHz which means we can use the same values of PSC and Arr calculated in part 1.

```

//Task 3
static volatile int CountA = 0;
static volatile int CountB = 0;
static volatile int CountC = 0;

void HAL_TIM_PeriodElapsedCallback( TIM_HandleTypeDef *htim){
    if (htim->Instance == TIM2) {
        CountA ++;
        CountB ++;
        CountC ++;

        if (CountA >= 500) {
            CountA = 0;
            HAL_GPIO_TogglePin(GPIOE, GPIO_PIN_10);
        }

        if (CountB >= 200) {
            CountB = 0;
            HAL_GPIO_TogglePin(GPIOE, GPIO_PIN_11);
        }

        if (CountC >= 100) {
            CountC = 0;
            HAL_GPIO_TogglePin(GPIOE, GPIO_PIN_12);
        }
    }
}

```

```
/* USER CODE END TIM3_Init 1 */  
htim3.Instance = TIM3;  
htim3.Init.Prescaler = 47;  
htim3.Init.CounterMode = TIM_COUNTERMODE_UP;  
htim3.Init.Period = 999;  
htim3.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;  
htim3.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_DISABLE;  
if (HAL_TIM_Base_Init(&htim3) != HAL_OK)  
{
```

Task 3 Extension:

we Are using TIM3 CH1, Pin is PC6